Reinforcement Learning Notes

Minghao Zhao

Spring 2026

Updated on: 14th April 2026

Instructor: Dr. Shiyu Zhao
Institution: University of California, San Diego

Contents

§1 Basic Concepts

This chapter introduces the fundamental building blocks of reinforcement learning (RL). The goal of RL is to find the *most optimal policy* for an agent interacting with an environment. The key framework used is the **Markov Decision Process (MDP)**.

§1.1 State and Action

Definition. *A **state** s describes the current status of the agent with respect to the environment. The set of all possible states is called the **state space**, denoted $\mathcal{S}$.*

Definition. *An **action** a is a choice available to the agent at a given state. The set of all possible actions is called the **action space**, denoted $\mathcal{A}$. Different states may have different action spaces, written $\mathcal{A}(s)$.*

Examples. *In a 3×3 grid-world, there are nine states $s_1, \ldots, s_9$ and five actions a_1 (up), a_2 (right), a_3 (down), a_4 (left), a_5 (stay). Some cells are forbidden (negative reward) and one is the target (positive reward).*

§1.2 State Transition

When an agent takes action a in state s, it may move to a new state s'. This is called a **state transition**.

Definition. *The **state transition probability** $p(s' \mid s, a)$ is the probability that the agent moves to state s' after taking action a in state s. It satisfies*

$$\sum_{s' \in \mathcal{S}} p(s' \mid s, a) = 1 \quad \forall\, (s, a).$$

For a deterministic transition, $p(s' \mid s, a) = 1$ for exactly one s' and 0 otherwise. In general, transitions can be stochastic (e.g. due to wind gusts).

Remark. When the agent attempts to move beyond the grid boundary, it is bounced back to the same state. Forbidden cells are treated as accessible but with a penalty reward.

§1.3 Policy

Definition. *A **policy** π tells the agent which action to take at every state. Formally, $\pi(a \mid s)$ is the probability of taking action a in state s, satisfying $\sum_{a \in \mathcal{A}(s)} \pi(a \mid s) = 1$ for all s.*

Policies can be:

- **Deterministic**: $\pi(a^* \mid s) = 1$ for a single action a^* at each state.
- **Stochastic**: positive probability is assigned to multiple actions.

§1.4 Reward

Definition. *A **reward** R_{t+1} is a scalar signal received by the agent after taking action A_t in state S_t and transitioning to state S_{t+1}. The reward probability is $p(r \mid s, a)$, satisfying $\sum_r p(r \mid s, a) = 1$.*

Rewards encode the goal:

- Positive reward encourages the action; negative reward discourages it.
- Only *relative* reward values matter—adding a constant to all rewards does not change the optimal policy.
- A typical grid-world setting: $r_{\text{boundary}} = -1$, $r_{\text{forbidden}} = -1$, $r_{\text{target}} = +1$, $r_{\text{other}} = 0$.

§1.5 Trajectories, Returns, and Episodes

Starting from state $S_t = s$ and following policy π, the agent generates a **trajectory**:

$$S_t \xrightarrow{A_t} S_{t+1}, R_{t+1} \xrightarrow{A_{t+1}} S_{t+2}, R_{t+2} \xrightarrow{A_{t+2}} \cdots$$

Definition. *The **discounted return** G_t starting at time t is*

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1},$$

*where $\gamma \in (0, 1)$ is the **discount rate**.*

The discount rate serves two purposes:

1. Ensures convergence of infinite-horizon sums.
2. Controls myopia: $\gamma \to 0$ emphasises near-future rewards (short-sighted); $\gamma \to 1$ emphasises far-future rewards (far-sighted).

A useful recursive relation (bootstrapping identity):

$$G_t = R_{t+1} + \gamma G_{t+1}.$$

Definition. *An **episode** is a trajectory that terminates at a terminal state. Tasks with episodes are **episodic tasks**; tasks with no terminal state are **continuing tasks**.*

§1.6 Markov Decision Processes

All concepts above are unified in the MDP framework.

Definition. *A **Markov Decision Process (MDP)** is a tuple $(\mathcal{S}, \mathcal{A}, p(s' \mid s, a), p(r \mid s, a), \gamma)$ equipped with the **Markov property**:*

$$p(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \ldots) = p(s_{t+1} \mid s_t, a_t).$$

The next state and reward depend only on the current state and action, not the history.

Remark. Once a policy is fixed in an MDP, it degenerates into a **Markov process (Markov chain)**.

§2 State Values and Bellman Equation

§2.1 State Value

Because G_t is a random variable (both policy and model may be stochastic), we use its expected value to evaluate a policy.

Definition. *The **state-value function** (or **state value**) of policy π is*

$$v_\pi(s) = \mathbb{E}[G_t \mid S_t = s], \quad s \in \mathcal{S}.$$

- $v_\pi(s)$ depends on s (conditional expectation) and on π (trajectories are generated under π).
- $v_\pi(s)$ does *not* depend on t for stationary models.
- A policy π_1 is **better** than π_2 if $v_{\pi_1}(s) \geq v_{\pi_2}(s)$ for all s.

§2.2 Bellman Equation

Using the bootstrapping identity $G_t = R_{t+1} + \gamma G_{t+1}$:

$$\begin{aligned} v_\pi(s) &= \mathbb{E}[G_t \mid S_t = s] \\ &= \mathbb{E}[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\ &= \mathbb{E}[R_{t+1} \mid S_t = s] + \gamma\, \mathbb{E}[G_{t+1} \mid S_t = s]. \end{aligned}$$

Expanding each term (using the law of total expectation and the Markov property):

$$\mathbb{E}[R_{t+1} \mid S_t = s] = \sum_a \pi(a \mid s) \sum_r p(r \mid s, a)\, r,$$

$$\mathbb{E}[G_{t+1} \mid S_t = s] = \sum_{s'} v_\pi(s') \sum_a \pi(a \mid s)\, p(s' \mid s, a).$$

Combining:

Definition (Bellman Equation)**.**

$$\boxed{v_\pi(s) = \sum_a \pi(a \mid s) \left[\sum_r p(r \mid s, a)\, r + \gamma \sum_{s'} p(s' \mid s, a)\, v_\pi(s') \right], \quad \forall\, s \in \mathcal{S}.}$$

This is a *linear* system of $|\mathcal{S}|$ equations in $|\mathcal{S}|$ unknowns. Solving it for a given π is called **policy evaluation**.

§2.3 Matrix-Vector Form

Collecting states $s_1, \ldots, s_n$ into vectors and matrices:

$$\mathbf{v}_\pi = \begin{pmatrix} v_\pi(s_1) \\ \vdots \\ v_\pi(s_n) \end{pmatrix}, \quad [\mathbf{r}_\pi]_s = \sum_a \pi(a \mid s) \sum_r p(r \mid s, a)\, r,$$

$$[P_\pi]_{s,s'} = \sum_a \pi(a \mid s)\, p(s' \mid s, a).$$

The Bellman equation becomes:

$$\mathbf{v}_\pi = \mathbf{r}_\pi + \gamma P_\pi \mathbf{v}_\pi.$$

§2.4 Solving the Bellman Equation

Method 1 (closed-form):

$$\mathbf{v}_\pi = (I - \gamma P_\pi)^{-1}\mathbf{r}_\pi.$$

The matrix $I - \gamma P_\pi$ is always invertible because the eigenvalues of γP_π all have magnitude $\leq \gamma < 1$.

Method 2 (iterative): Starting from any initial guess $\mathbf{v}^{(0)}$, iterate:

$$\mathbf{v}^{(j+1)} = \mathbf{r}_\pi + \gamma P_\pi \mathbf{v}^{(j)}, \quad j = 0, 1, 2, \ldots$$

This converges to the unique solution $\mathbf{v}_\pi$ because $f(\mathbf{v}) = \mathbf{r}_\pi + \gamma P_\pi \mathbf{v}$ is a contraction mapping with constant γ.

§2.5 Action Value

Definition. *The **action-value function** (or **action value / Q-value**) is*

$$q_\pi(s, a) = \mathbb{E}[G_t \mid S_t = s,\ A_t = a].$$

Relationship between state values and action values:

1. A state value is the π-weighted average of action values:

$$v_\pi(s) = \sum_a \pi(a \mid s)\, q_\pi(s, a).$$

2. An action value can be expressed in terms of next-state values:

$$q_\pi(s, a) = \sum_r p(r \mid s, a)\, r + \gamma \sum_{s'} p(s' \mid s, a)\, v_\pi(s').$$

Remark. Even if policy π never selects action a at state s, the action value $q_\pi(s, a)$ is still well-defined: it measures the return obtained by taking a once and then following π thereafter. This is crucial for policy improvement.

§3 Optimal State Values and Bellman Optimality Equation

§3.1 Optimal Policies and Optimal State Values

Definition. *A policy π^* is **optimal** if $v_{\pi^*}(s) \geq v_\pi(s)$ for all $s \in \mathcal{S}$ and for every other policy π. The corresponding state values $v^*(s) = v_{\pi^*}(s)$ are called **optimal state values**.*

Key facts (proven in the textbook):

- An optimal policy *always exists* (for finite MDPs).
- It may not be unique, but all optimal policies share the *same* optimal state values v^*.
- There always exists a *deterministic* optimal policy.

§3.2 Bellman Optimality Equation

The **Bellman Optimality Equation (BOE)** is obtained by replacing the policy-weighted sum with a maximisation:

$$\boxed{v(s) = \max_{\pi(s)} \sum_a \pi(a \mid s)\, q(s,a) = \max_{a \in \mathcal{A}(s)} q(s,a)}, \quad \forall\, s,$$

where

$$q(s,a) = \sum_r p(r \mid s,a)\, r + \gamma \sum_{s'} p(s' \mid s,a)\, v(s').$$

Here $v(s)$ is the *unknown* (to be solved). The maximisation is achieved by the greedy deterministic policy:

$$\pi^*(a \mid s) = \begin{cases} 1, & a = a^*(s) = \arg\max_a\, q(s,a), \\ 0, & \text{otherwise.} \end{cases}$$

Matrix-vector form. Define $f(\mathbf{v}) = \max_{\pi \in \Pi}(\mathbf{r}_\pi + \gamma P_\pi \mathbf{v})$, where the max is taken element-wise. Then the BOE reads:

$$\mathbf{v} = f(\mathbf{v}).$$

§3.3 Contraction Mapping Theorem

Definition. *A function $f : \mathbb{R}^d \to \mathbb{R}^d$ is a **contraction mapping** if there exists $\gamma \in (0,1)$ such that*

$$\|f(\mathbf{x}_1) - f(\mathbf{x}_2)\| \le \gamma\, \|\mathbf{x}_1 - \mathbf{x}_2\| \quad \forall\, \mathbf{x}_1, \mathbf{x}_2 \in \mathbb{R}^d.$$

*A point $\mathbf{x}^*$ with $f(\mathbf{x}^*) = \mathbf{x}^*$ is called a **fixed point**.*

Theorem 3.1 (Contraction Mapping / Banach Fixed-Point Theorem)**.** *If f is a contraction mapping, then:*

1. ***Existence**: A fixed point $\mathbf{x}^*$ exists.*
2. ***Uniqueness**: The fixed point is unique.*
3. ***Algorithm**: Starting from any $\mathbf{x}_0$, the iterates $\mathbf{x}_{k+1} = f(\mathbf{x}_k)$ converge to $\mathbf{x}^*$ exponentially fast.*

It can be shown that $f(\mathbf{v}) = \max_\pi(\mathbf{r}_\pi + \gamma P_\pi \mathbf{v})$ is a contraction mapping (with respect to the ℓ^∞ norm) with constant γ. Therefore the BOE has a unique solution v^*, and the iterative algorithm $\mathbf{v}_{k+1} = f(\mathbf{v}_k)$ converges to v^* from any starting point.

Remark (Optimal action value)*.* The optimal action value is defined as

$$q^*(s,a) = \sum_r p(r \mid s,a)\, r + \gamma \sum_{s'} p(s' \mid s,a)\, v^*(s').$$

Then $v^*(s) = \max_a q^*(s,a)$ and the optimal policy satisfies $\pi^*(a \mid s) = 1$ iff $a = \arg\max_a q^*(s,a)$.

§4 Value Iteration and Policy Iteration

The algorithms in this chapter are **model-based** (they require the transition and reward probabilities) and fall under the umbrella of *dynamic programming*.

§4.1 Value Iteration

Value iteration directly applies the contraction mapping iteration to the BOE.

Algorithm. Starting from any $\mathbf{v}_0$, repeat for $k = 0, 1, 2, \ldots$:

1. **Policy update** — compute greedy action values and policy:

$$q_k(s,a) = \sum_r p(r \mid s,a)\, r + \gamma \sum_{s'} p(s' \mid s,a)\, v_k(s'),$$

$$a_k^*(s) = \arg\max_a q_k(s,a), \qquad \pi_{k+1}(a \mid s) = \begin{cases} 1 & a = a_k^*(s) \\ 0 & \text{else} \end{cases}.$$

2. **Value update**:

$$v_{k+1}(s) = \max_a q_k(s,a) = q_k(s, a_k^*(s)), \quad \forall s.$$

Convergence is guaranteed by the contraction mapping theorem: $v_k \to v^*$ and $\pi_k \to \pi^*$.

Remark. The intermediate values v_k are *not* state values (they do not satisfy the Bellman equation of any policy in general); they are numerical approximations converging to v^*.

§4.2 Policy Iteration

Policy iteration alternates between two steps, each starting from a policy π_k.

§4.2.1 Step 1: Policy Evaluation

Solve the Bellman equation for v_{π_k}:

$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}.$$

This is done by the iterative algorithm (initialised at any $v^{(0)}$):

$$v^{(j+1)}(s) = \sum_a \pi_k(a \mid s) \left[\sum_r p(r \mid s,a)\, r + \gamma \sum_{s'} p(s' \mid s,a)\, v^{(j)}(s') \right],$$

until convergence (i.e. $\left\| v^{(j+1)} - v^{(j)} \right\| < \varepsilon$).

§4.2.2 Step 2: Policy Improvement

Given v_{π_k}, compute action values and update the policy greedily:

$$q_{\pi_k}(s,a) = \sum_r p(r \mid s,a)\, r + \gamma \sum_{s'} p(s' \mid s,a)\, v_{\pi_k}(s'),$$

$$\pi_{k+1}(a \mid s) = \begin{cases} 1 & a = \arg\max_{a'} q_{\pi_k}(s,a') \\ 0 & \text{otherwise} \end{cases}.$$

Lemma 4.1 (Policy Improvement)**.** *The improved policy satisfies $v_{\pi_{k+1}} \geq v_{\pi_k}$ (element-wise).*

Theorem 4.2 (Convergence of Policy Iteration)**.** *The state-value sequence $\{v_{\pi_k}\}$ generated by policy iteration converges to the optimal state value v^*, and the corresponding policies converge to an optimal policy.*

Remark. The intermediate values in policy iteration *are* state values: each v_{π_k} satisfies the Bellman equation of π_k.

§4.3 Truncated Policy Iteration

Policy iteration embeds an infinite iterative process inside the policy evaluation step. In practice, we run only a *finite* number j_{truncate} of evaluation iterations, yielding **truncated policy iteration**.

Relationship to other algorithms:

- $j_{\text{truncate}} = 1$: equivalent to **value iteration**.
- $j_{\text{truncate}} = \infty$: equivalent to **policy iteration**.
- $1 < j_{\text{truncate}} < \infty$: strictly in between, converging faster than value iteration but more computationally light than full policy iteration.

Pseudocode. Initialise π_0 and v_0. For $k = 0, 1, 2, \ldots$:

1. *Policy evaluation (truncated):* set $v_k^{(0)} = v_{k-1}$ and iterate

$$v_k^{(j+1)}(s) = \sum_a \pi_k(a \mid s) \left[\sum_r p(r \mid s, a)\, r + \gamma \sum_{s'} p(s' \mid s, a)\, v_k^{(j)}(s') \right]$$

   for $j = 0, \ldots, j_{\text{truncate}} - 1$. Set $v_k = v_k^{(j_{\text{truncate}})}$.

2. *Policy improvement:* for every s,

$$\pi_{k+1}(a \mid s) = \begin{cases} 1 & a = \arg\max_{a'} q_k(s, a') \\ 0 & \text{else} \end{cases},$$

   where $q_k(s, a) = \sum_r p(r \mid s, a)\, r + \gamma \sum_{s'} p(s' \mid s, a)\, v_k(s')$.

Remark. The general guideline is to run a *small but greater than one* number of iterations in the policy evaluation step. A few iterations already speed up convergence significantly over value iteration.

Generalized Policy Iteration. The idea of iterating between value and policy updates is called **generalized policy iteration (GPI)** and underpins virtually all RL algorithms.

§5 Monte Carlo Methods

The algorithms in the previous chapter required the system model $\{p(r \mid s, a),\, p(s' \mid s, a)\}$. Monte Carlo (MC) methods are the first **model-free** algorithms: they learn from experience samples (episodes) rather than a known model.

Philosophy: if no model is available, collect data; estimate expectations from sample averages.

§5.1 Mean Estimation and the Law of Large Numbers

State and action values are *expected returns*—i.e. means of random variables. Estimating them from samples is a **mean estimation** problem.

Fact (Law of Large Numbers)**.** *Let $\{x_i\}_{i=1}^n$ be i.i.d. samples of random variable X. Define $\bar{x} = \frac{1}{n}\sum_{i=1}^n x_i$. Then:*

$$\mathbb{E}[\bar{x}] = \mathbb{E}[X], \qquad \text{Var}[\bar{x}] = \frac{\text{Var}[X]}{n} \xrightarrow{n\to\infty} 0.$$

That is, $\bar{x}$ is an unbiased estimator of $\mathbb{E}[X]$, and as $n \to \infty$, $\bar{x} \to \mathbb{E}[X]$ almost surely.

Remark. Samples must be **independent and identically distributed (i.i.d.)** for the law of large numbers to hold.

§5.2 Converting Policy Iteration to Model-Free: The MC Idea

In policy iteration, the model is used in two places:

1. *Policy evaluation*: solve $v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$ (requires model).
2. *Policy improvement*: compute $q_{\pi_k}(s,a) = \sum_r p(r \mid s,a)\, r + \gamma \sum_{s'} p(s' \mid s,a)\, v_{\pi_k}(s')$ (requires model).

The key insight is that action values can be estimated *directly* from samples without a model:

$$q_{\pi_k}(s,a) = \mathbb{E}[G_t \mid S_t = s,\ A_t = a] \approx \frac{1}{n} \sum_{i=1}^{n} g_{\pi_k}^{(i)}(s,a),$$

where $g_{\pi_k}^{(i)}(s,a)$ is the discounted return of the i-th episode starting from (s,a) and following π_k thereafter. As $n \to \infty$, this approximation becomes exact.

Remark. Model-free algorithms must estimate **action values** directly (not state values), because converting state values to action values via (5.1) would still require the model.

§5.3 MC Basic Algorithm

MC Basic is the simplest MC algorithm: it is exactly the policy iteration algorithm with the model-based policy evaluation replaced by the MC sample-average estimate of action values.

Algorithm (MC Basic). Initialise policy π_0. For $k = 0, 1, 2, \ldots$:

1. **Policy evaluation (MC):** for every $(s,a) \in \mathcal{S} \times \mathcal{A}$, collect n episodes starting from (s,a) and following π_k. Approximate:

$$q_{\pi_k}(s,a) \approx q_k(s,a) = \frac{1}{n} \sum_{i=1}^{n} g_{\pi_k}^{(i)}(s,a).$$

2. **Policy improvement:** update greedily,

$$a_k^*(s) = \arg\max_a q_k(s,a), \qquad \pi_{k+1}(a \mid s) = \begin{cases} 1 & a = a_k^*(s) \\ 0 & \text{else} \end{cases}.$$

Convergence. When n is sufficiently large, $q_k(s,a) \approx q_{\pi_k}(s,a)$ accurately, and MC Basic has the same convergence guarantees as policy iteration. In practice, even with a finite n, the algorithm typically converges—analogous to truncated policy iteration, which also uses approximate action values.

Limitations. MC Basic is too sample-inefficient for practical use:

- It requires *infinitely long* episodes to accurately approximate action values when trajectories do not terminate quickly.
- It collects a fresh batch of n episodes for *every* (s,a) in every iteration—a prohibitive number of samples.
- Sparse rewards make value estimation difficult with short episodes.

Nevertheless, MC Basic is conceptually important as the foundation for more efficient MC algorithms (MC Exploring Starts and MC ε-Greedy).